

TECH

FINTECH SCENARIOS

FINTECH SCENARIOS

FINTECH SCENARIOS

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК общая практика fintech-фронтенда

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 ТЯЖЁЛЫЙ	ВЫСОКОЧАСТОТНЫЕ ЦЕНЫ DELAY Q1 · Q2 · Q3 · Q4	STALE DATA	ВЫСОКОЧАСТОТНЫЕ ЦЕНЫ DELAY	STALE DATA
ДЕНЬ 2 СРЕДНИЙ	AUDITABILITY Q5 · Q6	SENSITIVE LOGGING	AUDITABILITY	SENSITIVE LOGGING
ДЕНЬ 3 ТЯЖЁЛЫЙ	TRADING FORM NG TX Q7 · Q8 · Q9	DOUBLE SUBMIT	PENDING TX	DOUBLE SUBMIT PENDING TX
ДЕНЬ 4 ТЯЖЁЛЫЙ	IDEMPOTENCY Q10 · Q11	INCONSISTENT BACKEND	IDEMPOTENCY	INCONSISTENT BACKEND
ДЕНЬ 5 ТЯЖЁЛЫЙ	RBAC KILL SWITCH Q12 · Q13 · Q14	HIGH-RISK TESTING	RBAC KILL SWITCH	HIGH-RISK TESTING
ДЕНЬ 6 СРЕДНИЙ	ПОСТ-РЕЛИЗ MONITORING BACK PARTIAL OUTAGE Q15 · Q16 · Q17	DEGRADED FALL	ПОСТ-РЕЛИЗ MONITORING BACK PARTIAL OUTAGE	DEGRADED FALL
ДЕНЬ 7 СРЕДНИЙ	PRECISION/ROUNDING TIMEZONE Q18 · Q19 · Q20	MONEY FORMATTING	PRECISION/ROUNDING TIMEZONE	MONEY FORMATTING

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ТЯЖЁЛЫЙ 4 ВОПРОСА

ВЫСОКОЧАСТОТНЫЕ ЦЕНЫ STALE DATA DELAY

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ВЫСОКОЧАСТОТНЫЕ ЦЕНЫ STALE DATA DELAY

ВОПРОСЫ ДНЯ

- Q01 Как проектировать UI, где цена меняется много раз в секунду?
- Q02 Как не показывать пользователю устаревшие финансовые данные?
- Q03 Как обозначить stale market data?
- Q04 Как обрабатывать задержку real-time feed?

Как проектировать UI, где цена меняется много раз в секунду?

ОПИСАНИЕ

Батчить апдейты (накопить N мс или rAF-flush), диффовать по id, виртуализировать список, виджеты независимы. Не делать setState на каждое сообщение. Для тяжёлых трансформаций – Web Worker. Подсветка изменения flash-эффектом без layout-shift.

КАК РАБОТАЕТ

- 01 Накапливать в Map → flush через rAF.
- 02 useSyncExternalStore + селекторы.
- 03 Виртуализация (TanStack Virtual).
- 04 Composite-only анимации (transform, opacity).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Биржевые стаканы.
- 02 Ticker-панели.
- 03 Real-time дашборды.

ПОДВОДНЫЕ КАМНИ

- 01 setState per tick = render storm.
- 02 Layout-shift при flash → CLS.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tearing?
- 02 Когда canvas лучше DOM?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ВОПРОС Q02

Как не показывать пользователю устаревшие финансовые данные?

ОПИСАНИЕ

Метка timestamp последнего апдейта, мониторинг age данных, показывать индикатор stale (серый цвет / иконка), запрет торговать на stale-данных, fallback на «—» если данных нет N секунд.

КАК РАБОТАЕТ

- 01 lastUpdate timestamp в state.
- 02 interval-проверка age > threshold.
- 03 stale-флаг → визуальный индикатор.
- 04 Disable trade-кнопок при stale.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading dashboards.
- 02 Котировочные панели.

ПОДВОДНЫЕ КАМНИ

- 01 Без timestamp – пользователь не знает.
- 02 Disabled без подсказки → жалобы.

МОГУТ ДОСПРОСИТЬ

- 01 Что показывать при degraded?
- 02 Как проверить age в realtime?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/inp>

Как обозначить stale market data?

ОПИСАНИЕ

Визуально: иконка / opacity / outline. Текстом: «обновлено N секунд назад». Цвет – серый (не сигнализирующий). Блокировать actions, требующие актуальных данных. Aria-live для assistive tech.

КАК РАБОТАЕТ

- 01 Иконка clock + tooltip 'обновлено N сек назад'.
- 02 Opacity 0.5 для stale-блока.
- 03 Disable trade buttons.
- 04 aria-live="polite" для смены статуса.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Live-prices.
- 02 Order book.
- 03 Position summaries.

ПОДВОДНЫЕ КАМНИ

- 01 Только цвет – colorblind.
- 02 Без блокировки actions – риски.

МОГУТ ДОСПРОСИТЬ

- 01 Что аккуратно блокировать?
- 02 Как тестировать stale UX?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-live>

ВОПРОС Q04

Как обрабатывать задержку real-time feed?

ОПИСАНИЕ

Измерять latency (timestamp source vs receive vs paint), показывать индикатор задержки, при growing latency – fallback на polling, alerting в RUM. Метрика – P95/P99 tick-to-paint.

КАК РАБОТАЕТ

- 01 tick.ts → recv.ts → paint.ts → diff = latency.
- 02 RUM: histogram latency.
- 03 Threshold → degraded mode.
- 04 Polling-fallback при долгой задержке.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading-системы.
- 02 Live-charts.
- 03 Multiplayer.

ПОДВОДНЫЕ КАМНИ

- 01 Без измерения – не знаем что течёт.
- 02 User видит застывшие цифры без объяснения.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tick-to-paint?
- 02 Как реализовать degraded?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/inp>

AUDITABILITY

SENSITIVE LOGGING

ЧТО УЧИМ В ЭТОТ ДЕНЬ

AUDITABILITY SENSITIVE LOGGING

ВОПРОСЫ ДНЯ

Q05 Как проектировать auditability на frontend?

Q06 Как не логировать sensitive financial data?

ВОПРОС Q05

Как проектировать auditability на frontend?

ОПИСАНИЕ

Логировать ключевые user-actions: clicks по trade, отправки форм, navigation. Включать context (user-id, timestamp, version). Хранить локально и/или отправлять в audit-log на сервер. Не логировать PII / sensitive financial data.

КАК РАБОТАЕТ

- 01 Event log: action, ts, user, ctx.
- 02 sendBeacon для надёжной доставки.
- 03 Server audit-log с retention.
- 04 Without PII / passwords / pan.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Brokerage / banking.
- 02 Compliance-требования.

ПОДВОДНЫЕ КАМНИ

- 01 Лог PII = legal-проблема.
- 02 Без version – не воспроизвести.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое immutable audit-log?
- 02 Как маскировать sensitive?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-top-ten/A09_2021-Security_Logging_and_Monitoring_Failures

Как не логировать sensitive financial data?

ОПИСАНИЕ

Маска / redact на уровне log-helper: PAN, CVV, account numbers, balance, password – never. Whitelist полей вместо blacklist. Тесты на отсутствие sensitive в payload. Sentry beforeSend фильтрация.

КАК РАБОТАЕТ

- 01 Redact на уровне logger.
- 02 Whitelist полей.
- 03 Sentry beforeSend filter.
- 04 Code-review правило.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая financial app.
- 02 Healthcare.
- 03 GDPR-юрисдикция.

ПОДВОДНЫЕ КАМНИ

- 01 Stack trace может содержать data.
- 02 Console.log в production.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое PCI DSS?
- 02 Как фильтровать Sentry events?

ПОЛНАЯ СТАТЬЯ

<https://docs.sentry.io/platforms/javascript/configuration/options/#beforeSend>

ДЕНЬ 3 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

TRADING FORM

DOUBLE SUBMIT

PENDING TX

ЧТО УЧИМ В ЭТОТ ДЕНЬ

TRADING FORM DOUBLE SUBMIT PENDING TX

ВОПРОСЫ ДНЯ

Q07 Как проектировать безопасный trading form?

Q08 Как защититься от double submit order?

Q09 Как показывать pending transaction state?

Как проектировать безопасный trading form?

ОПИСАНИЕ

Подтверждение через modal (review order перед submit), явный лимит-индикатор, защита от double-submit (см. Q8), idempotency-key в запросе, optimistic UI с откатом, audit-log каждого action.

КАК РАБОТАЕТ

- 01 Двухшаговая форма: ввод → review → confirm.
- 02 Idempotency-Key в HTTP header.
- 03 Disable submit во время pending.
- 04 Audit-log каждого confirm.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Buy / Sell orders.
- 02 Wire transfers.
- 03 Crypto-tx.

ПОДВОДНЫЕ КАМНИ

- 01 Один-клик submit без confirm – большой риск.
- 02 Без idempotency дубль = дубль ордера.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Idempotency-Key?
- 02 Как откатывать failed order?

ПОЛНАЯ СТАТЬЯ

https://stripe.com/docs/api/idempotent_requests

ВОПРОС Q08

Как защититься от double submit order?

ОПИСАНИЕ

На client: disable submit + isSubmitting флаг. На сервере: idempotency-key (генерируем при первой попытке, переиспользуем при retry). Server возвращает идентичный ответ для того же ключа.

КАК РАБОТАЕТ

- 01 isSubmitting → disabled button.
- 02 Idempotency-Key UUID, генерируется при mount формы.
- 03 Server: store key+result для retry.
- 04 Локальный флаг + setTimeout cleanup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая mutation с побочным эффектом.
- 02 Особенно – финансы.

ПОДВОДНЫЕ КАМНИ

- 01 Только client-disable – refresh обходит.
- 02 Без unique constraint на BE → дубль.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое idempotency на сервере?
- 02 Как генерировать ключ?

ПОЛНАЯ СТАТЬЯ

https://stripe.com/docs/api/idempotent_requests

Как показывать pending transaction state?

ОПИСАНИЕ

DU состояний: idle/submitting/pending/confirmed/failed. UI: spinner + сообщение 'обрабатывается', disabled retry, при confirmed – success, при failed – error с действиями (retry/support). Не оставлять пользователя в неопределённости.

КАК РАБОТАЕТ

- 01 DU pending vs confirmed.
- 02 Polling status / WS push.
- 03 Visible message 'обрабатывается, не закрывайте'.
- 04 Timeout fallback для long-pending.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Crypto / blockchain TX.
- 02 Bank transfers.
- 03 Complex orders.

ПОДВОДНЫЕ КАМНИ

- 01 Failed без действий – пользователь застрял.
- 02 Pending не покажется при reload.

МОГУТ ДОСПРОСИТЬ

- 01 Как восстановить state после reload?
- 02 Что показать при timeout?

ПОЛНАЯ СТАТЬЯ

<https://stately.ai/docs>

ДЕНЬ 4 НАГРУЗКА ТЯЖЁЛЫЙ 2 ВОПРОСА

IDEMPOTENCY INCONSISTENT BACKEND

ЧТО УЧИМ В ЭТОТ ДЕНЬ

IDEMPOTENCY INCONSISTENT BACKEND

ВОПРОСЫ ДНЯ

Q10 Как проектировать idempotent frontend flow?

Q11 Как обрабатывать inconsistent backend state?

Как проектировать idempotent frontend flow?

ОПИСАНИЕ

Каждая mutation имеет stable Idempotency-Key (UUID на mount формы). При retry – тот же ключ. Server возвращает кэшированный результат для того же ключа. UX: при ретрае не создаётся дубль.

КАК РАБОТАЕТ

- 01 useRef(uuid) на mount.
- 02 Header 'Idempotency-Key': key в каждом submit-call.
- 03 Retry с тем же ключом.
- 04 On success → cleanup ref для следующей формы.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Submit form / order / transfer.
- 02 Network-flaky сценарии.

ПОДВОДНЫЕ КАМНИ

- 01 Новый ключ на каждый retry → дубль.
- 02 Server не реализует idempotency → бесполезно.

МОГУТ ДОСПРОСИТЬ

- 01 Что должно быть idempotent на BE?
- 02 Как генерировать ключ?

ПОЛНАЯ СТАТЬЯ

https://stripe.com/docs/api/idempotent_requests

ВОПРОС Q11

Как обрабатывать inconsistent backend state?

ОПИСАНИЕ

Backend временно показывает inconsistent: order создан, но balance не списан. Решения: показывать pending state, polling до consistent, версионирование state, manual reconciliation UI, support-flow.

КАК РАБОТАЕТ

- 01 Pending-state до eventual consistency.
- 02 Polling статуса каждые N секунд.
- 03 Version/etag для отслеживания.
- 04 Refresh-button + 'обратиться в support'.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Distributed backends.
- 02 Multi-step transactions.

ПОДВОДНЫЕ КАМНИ

- 01 Показать confirmed без confirm-сигнала – false success.
- 02 Без polling – не дожждётся.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое eventual consistency?
- 02 Когда показать support-link?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/patterns-of-distributed-systems/eventual-consistency.html>

RBAC

KILL SWITCH

HIGH-RISK TESTING

ЧТО УЧИМ В ЭТОТ ДЕНЬ

RBAC KILL SWITCH HIGH-RISK TESTING

ВОПРОСЫ ДНЯ

Q12 Как проектировать role-based access для финансовых операций?

Q13 Как проектировать kill switch для рискованной фичи?

Q14 Как тестировать high-risk UI перед релизом?

ВОПРОС Q12

Как проектировать role-based access для финансовых операций?

ОПИСАНИЕ

RBAC: роли + permissions per action. Frontend: <Can action="...">, useCan-hook, route-guards. Backend проверяет всегда – frontend только UX. Audit-log смены ролей. Принцип наименьших привилегий.

КАК РАБОТАЕТ

- 01 useCan('order.create') – хук.
- 02 <Can action="order.create">.
- 03 ProtectedRoute по роли.
- 04 Server validates на каждом action.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-role broker UI.
- 02 B2B-инструменты.
- 03 Admin-панели.

ПОДВОДНЫЕ КАМНИ

- 01 Только client-check = security hole.
- 02 Permissions stale при смене роли.

МОГУТ ДОСПРОСИТЬ

- 01 RBAC vs ABAC?
- 02 Как обновлять permissions без relogin?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

Как проектировать kill switch для рискованной фичи?

ОПИСАНИЕ

Feature flag на server-side: можно мгновенно выключить без redeploy. Client polls / WS-listens на изменения. UX: graceful degradation, не сломанный экран. Tested kill-switch – часть QA.

КАК РАБОТАЕТ

- 01 Server-side flag (LaunchDarkly / Unleash).
- 02 Client subscribe / poll.
- 03 Fallback UI 'фича временно недоступна'.
- 04 Тестировать kill-switch перед production.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 High-risk releases.
- 02 Trading-фичи.
- 03 Compliance-чувствительные.

ПОДВОДНЫЕ КАМНИ

- 01 Только client-side flag – ломает risk-mitigation.
- 02 Без UX-fallback – broken UI.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое gradual rollout?
- 02 Чем отличается kill switch от feature flag?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/feature-toggles.html>

В О П Р О С Q 1 4

Как тестировать high-risk UI перед релизом?**ОПИСАНИЕ**

Несколько слоёв: unit (логика расчётов), integration (форма + API), e2e (полный flow на staging), visual regression, accessibility, exploratory тестирование на staging с реальными данными, shadow-traffic / canary.

КАК РАБОТАЕТ

- 01 Unit-тесты на расчёты (precision).
- 02 Integration с MSW.
- 03 E2E на staging.
- 04 Visual regression (Chromatic).
- 05 Canary release с %-rollout.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading.
- 02 Payments.
- 03 Critical workflows.

ПОДВОДНЫЕ КАМНИ

- 01 Только unit без e2e – пропущенные integration-баги.
- 02 Без canary – все или никто.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое shadow traffic?
- 02 Чем canary от blue-green отличается?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/CanaryRelease.html>

ПОСТ-РЕЛИЗ MONITORING

DEGRADED FALLBACK

PARTIAL OUTAGE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ПОСТ-РЕЛИЗ MONITORING

DEGRADED FALLBACK

PARTIAL OUTAGE

ВОПРОСЫ ДНЯ

Q15 Как мониторить frontend после релиза trading-фичи?

Q16 Как проектировать fallback при degraded backend?

Q17 Как объяснить пользователю partial outage?

ВОПРОС Q15

Как мониторить frontend после релиза trading-фичи?

ОПИСАНИЕ

Метрики: error rate, JS errors, API failure rate, INP / LCP регрессии, business KPI (orders/min). Dashboards с release-annotations. Alerts на отклонения. Сравнение before/after релиза.

КАК РАБОТАЕТ

- 01 Sentry для JS errors.
- 02 RUM для CWV.
- 03 Business-metrics в dashboard.
- 04 Release-annotations on graphs.
- 05 Alerts на отклонение P95/P99.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Каждый production release.
- 02 Особенно – trading/payment.

ПОДВОДНЫЕ КАМНИ

- 01 Без annotation – невозможно связать с релизом.
- 02 Только error rate, без business – пропуск.

МОГУТ ДОСПРОСИТЬ

- 01 Какие frontend health checks нужны?
- 02 Что такое release annotation?

ПОЛНАЯ СТАТЬЯ

<https://docs.sentry.io/product/releases/>

Как проектировать fallback при degraded backend?

ОПИСАНИЕ

Cached данные, stale-while-revalidate, очередь для retry, read-only mode, понятное сообщение пользователю. Не показывать пустой экран. Различать частичный outage от полного.

КАК РАБОТАЕТ

- 01 react-query placeholderData.
- 02 Service Worker offline-cache.
- 03 Read-only banner.
- 04 User message: «временно недоступно X».

ГДЕ ВСТРЕЧАЕТСЯ

- 01 PWA с offline-mode.
- 02 Mobile-веб с нестабильной сетью.
- 03 Multi-service backends.

ПОДВОДНЫЕ КАМНИ

- 01 Stale без указания → false-comfort.
- 02 Без offline-cache reload даёт пусто.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое stale-while-revalidate?
- 02 Чем PWA помогает?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/offline-cookbook>

ВОПРОС Q17

Как объяснить пользователю partial outage?

ОПИСАНИЕ

Status banner: «N сервис временно недоступен», что работает / не работает. Disable affected actions. Link на status page. Real-time обновление статуса. Не оставлять догадываться.

КАК РАБОТАЕТ

- 01 Status banner на app-level.
- 02 Disable affected UI sections.
- 03 Link на /status.
- 04 Recover-notification после resolve.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SaaS / B2B-инструменты.
- 02 Multi-service products.

ПОДВОДНЫЕ КАМНИ

- 01 Без banner – пользователь думает «я сломал».
- 02 Без recover-уведомления – продолжает считать недоступным.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое status page?
- 02 Как auto-detect outage?

ПОЛНАЯ СТАТЬЯ

<https://www.atlassian.com/incident-management/handbook>

PRECISION/ROUNDING

MONEY FORMATTING

TIMEZONE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

PRECISION/ROUNDING

MONEY FORMATTING

TIMEZONE

ВОПРОСЫ ДНЯ

Q18 Как работать с precision/rounding в financial UI?

Q19 Как тестировать formatting денежных значений?

Q20 Как работать с timezone в финансовых данных?

ВОПРОС Q18

Как работать с precision/rounding в financial UI?

ОПИСАНИЕ

Никогда Number / float для денег. Использовать BigInt (целые копейки), decimal-библиотеки (decimal.js, dinero.js), string-представление по сети. Round по бизнес-правилу (banker's rounding, half-up). Тесты на edge cases.

КАК РАБОТАЕТ

- 01 Хранить копейки как integer / BigInt.
- 02 decimal.js / dinero.js для арифметики.
- 03 API передаёт строкой / cents.
- 04 Round-mode согласован с BE.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые расчёты денег.
- 02 Crypto (8 decimals).
- 03 FX-конвертации.

ПОДВОДНЫЕ КАМНИ

- 01 $0.1 + 0.2 \neq 0.3$ – IEEE 754.
- 02 Number.MAX_SAFE_INTEGER ограничен.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое banker's rounding?
- 02 Чем dinero.js полезен?

ПОЛНАЯ СТАТЬЯ

<https://dinerojs.com/>

Как тестировать `formatting` денежных значений?

ОПИСАНИЕ

Snapshot / fixture-based тесты на валюты (USD/EUR/RUB/JPY), negative values, разделитель тысяч, decimal places по валюте, локали. Property-based testing для random-значений. Использовать `Intl.NumberFormat`.

КАК РАБОТАЕТ

- 01 `Intl.NumberFormat({ style: 'currency', currency: 'USD' })`.
- 02 Тесты на edge: 0, негативное, очень большое.
- 03 Snapshot per currency.
- 04 Property-based с fast-check.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Финансовые UIs.
- 02 Multi-currency.
- 03 Bills / invoices.

ПОДВОДНЫЕ КАМНИ

- 01 Hardcoded \$ – не работает в EUR.
- 02 Подмена locale ломает разделители.

МОГУТ ДОСПРОСИТЬ

- 01 Что делает `Intl.NumberFormat`?
- 02 Чем property-based лучше example-based?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat

ВОПРОС Q20

Как работать с timezone в финансовых данных?

ОПИСАНИЕ

Сервер всегда в UTC, клиент конвертирует в local TZ. Хранение ISO-string с tz-offset (или UTC + tz-name). Library: Luxon / date-fns-tz. Edge-cases: DST, високосные секунды, разные trading sessions per market.

КАК РАБОТАЕТ

- 01 API: ISO 8601 с timezone.
- 02 Luxon / date-fns-tz / Temporal API.
- 03 Display в local TZ + индикатор TZ.
- 04 Trading hours per exchange.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-region trading.
- 02 Global SaaS.
- 03 Audit-logs.

ПОДВОДНЫЕ КАМНИ

- 01 `new Date(string)` интерпретирует разные форматы по-разному.
- 02 DST → '23:30' два раза в году.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Temporal API?
- 02 Чем Luxon от moment отличается?

ПОЛНАЯ СТАТЬЯ

<https://moment.github.io/luxon/>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Спроектировать UI с high-frequency tick + stale + delay (Q1-Q4)
- ☐ Описать auditability и sensitive logging (Q5, Q6)
- ☐ Спроектировать trading form + double submit + pending TX (Q7, Q8, Q9)
- ☐ Описать idempotent flow и обработку inconsistent backend (Q10, Q11)
- ☐ Описать RBAC + kill switch + high-risk testing (Q12, Q13, Q14)
- ☐ Описать post-release monitoring + degraded fallback + partial outage (Q15-Q17)
- ☐ Описать precision/rounding, money formatting, timezone (Q18-Q20)